IMD0029 - EDB1 - 2025.1 - Prova 4

Prof. Eiji Adachi

📝 LEIA ANTES DE COMEÇAR

- Atividade individual sem consulta a pessoas ou materiais (impresso ou eletrônico).
- O valor de cada questão está indicado no enunciado.
- Mantenha celulares e outros eletrônicos desligados durante a prova.
- Desvios éticos resultarão em nota zero nesta unidade.
- Você recebeu diretórios para cada questão, cada um contendo um código base, um arquivo `Makefile` e um teste executável, que pode ser executado com `make run-test`.
  ⚠ O teste não garante a corretude completa da implementação.
- **Não altere a assinatura das funções fornecidas.** Você pode criar funções auxiliares, mas a assinatura principal deve ser mantida.

📌 Critérios de Correção

Serão avaliados os seguintes pontos:

- Conformidade com as **assinaturas de função** e estrutura de diretórios.
- **Compilação limpa**, sem erros ou *warnings* (use o `Makefile`).
- **Correta execução dos programas** com os resultados esperados.
- **Complexidade** conforme especificado no enunciado.
- **Qualidade do código**: organização, indentação, nomes adequados, modularização, etc.

📦 Entregável

- Use a mesma estrutura de diretórios recebida, com os arquivos de solução em cada pasta de questão.
- O diretório raiz deve conter seu nome em letras maiúsculas no formato `PRIMEIRO_NOME_SOBRENOME`. Exemplo:

```
JOAO_SILVA
├── q1
├── q2
└── q3
```

- Compacte tudo em um `.zip` com o mesmo nome: `PRIMEIRO_NOME_SOBRENOME.zip`.
- ❌ Não inclua arquivos `.o` ou executáveis, mas mantenha os arquivos `makefile`.
- Entregue via SIGAA até o horário estabelecido. Atrasos só serão aceitos com justificativa válida (ex.: instabilidade no SIGAA).

Questão 1 - Valor: 2.0

Implemente a função abaixo, que busca a **primeira ocorrência** de um elemento em um array ordenado de inteiros:

```
int encontrarPrimeiroIndice(const int arr[], int tamanho, int alvo);
```

🔧 Comportamento esperado

- A função deve receber:
  - um array de inteiros ordenado em ordem crescente (`arr`),
  - o tamanho do array (`tamanho`),
  - e o valor a ser buscado (`alvo`).
- Ela deve retornar o **índice da primeira ocorrência** de `alvo` no array.
- Caso o elemento **não esteja presente**, deve retornar `-1`.

⏱️ Complexidade esperada

- O algoritmo deve ter complexidade de tempo **O(log n)**.

📌 Exemplo de uso

Array considerado: `{1, 2, 2, 2, 3, 4, 5}`

Chamada da função	Retorno
`encontrarPrimeiroIndice(arr, 7, 2);`	`1`
`encontrarPrimeiroIndice(arr, 7, 4);`	`5`
`encontrarPrimeiroIndice(arr, 7, 5);`	`6`
`encontrarPrimeiroIndice(arr, 7, 6);`	`-1`
`encontrarPrimeiroIndice(arr, 7, 1);`	`0`

📁 Onde implementar?

📄 Arquivo:

```
q1/src/PrimeiroIndice.cpp
```

🧪 Como testar?

No diretório `q1`, digite:

```
make run-test
```

Questão 2 - Valor: 2.0

Implemente o método abaixo na classe `Lista`, que representa uma **lista simplesmente encadeada com ponteiros para o primeiro e para o último nó da lista**:

```
int Lista::removerTodos(int val);
```

🔧 Comportamento esperado

- Recebe um valor inteiro `val` e remove **todos os nós da lista** com esse valor.
- A função deve retornar o número total de elementos removidos.
- ⏱️ Complexidade de tempo: **Θ(n)**
- ❌ Não usar estruturas auxiliares como `stack`, `array`, `vector`, etc.

📌 Exemplo de uso

Lista original: `{1 -> 7 -> 3 -> 7 -> 5 -> 7 -> 14}`

Chamada do método	Estado esperado após execução	Retorno
`removerTodos(7);`	`{1 -> 3 -> 5 -> 14}`	`3`
`removerTodos(20);`	`{1 -> 7 -> 3 -> 7 -> 5 -> 7 -> 14}`	`0`

📂 Onde implementar?

📄 Arquivo:

```
q2e3/src/Lista.cpp
```

🧪 Como testar?

No diretório `q2e3`, digite:

```
make run-test-remover
```

Questão 3 - Valor: 2.0

Implemente o método abaixo na classe `Lista`, que representa uma **lista simplesmente encadeada com ponteiros para o primeiro e para o último nó da lista**:

```
void Lista::inserirOrdenadoSemRepeticao(const std::string& elemento);
```

🔧 Comportamento esperado

- Recebe uma string `elemento` e tenta **inserir um novo nó** na lista de forma que ela permaneça **ordenada em ordem crescente**.
- A inserção deve manter a ordem crescente dos valores dos nós da lista.
- A inserção pode ocorrer:
  - no início da lista,
  - no meio,
  - ou ao final, dependendo do valor recebido.
- Se o valor **já estiver presente** na lista, **nenhum nó deve ser inserido** e a função deve retornar `false`.
- Caso contrário, o novo nó é inserido corretamente e a função retorna `true`.
- ⏱️ Complexidade de tempo: **Θ(n)** no pior caso.
- ❌ Não usar estruturas auxiliares como `stack`, `array`, `vector`, etc.

📌 Exemplo de uso

Lista original: `{2 -> 4 -> 7}`

Chamada do método	Estado esperado após execução	Retorno
`inserirOrdenado(1);`	`{1 -> 2 -> 4 -> 7}`	`true`
`inserirOrdenado(5);`	`{2 -> 4 -> 5 -> 7}`	`true`
`inserirOrdenado(9);`	`{2 -> 4 -> 7 -> 9}`	`true`
`inserirOrdenado(4);`	`{2 -> 4 -> 7}`	`false`

📁 Onde implementar?

📄 Arquivo:

```
q2e3/src/Lista.cpp
```

🧪 Como testar?

No diretório `q2e3`, digite:

```
make run-test-inserir
```

Questão 4 - Valor: 2.0

Explique como funciona o algoritmo de ordenação *quick sort*. Explique também em qual situação o *quick sort* cai em seu pior caso, explicitando qual sua complexidade neste caso.

Questão 5 - Valor: 2.0

Explique como funciona o algoritmo de ordenação *merge sort*. Explique também em qual situação o *merge sort* cai em seu pior caso, explicitando qual sua complexidade neste caso.